SQL
PROCEDURES

Ing. Ladislav Körösi, PhD.
ÚRK, FEI STU

ladislav.korosi@stuba.sk

PROCEDURES

“ A stored routine is a set of SQL statements that can be stored in the server.”

A stored procedure is a method to encapsulate repetitive tasks. They allow for variable declarations, flow control and other useful programming techniques.

PROCEDURES

Pros:

- **Share logic** with other applications. Stored procedures encapsulate functionality; this ensures that data access and manipulation are coherent between different applications.
- **Isolate** users from data tables. This gives you the ability to grant access to the stored procedures that manipulate the data but not directly to the tables.
- Provide a **security** mechanism. Considering the prior item, if you can only access the data using the stored procedures defined, no one else can execute a DELETE SQL statement and erase your data.
- To **improve performance** because it reduces network traffic. With a stored procedure, multiple calls can be melded into one.

PROCEDURES

Cons:

- **Increased load** on the database server -- most of the work is done on the server side, and less on the client side.
- There's a decent **learning curve**. You'll need to learn the syntax of MySQL statements in order to write stored procedures.
- You are **repeating the logic** of your application in two different places: your server code and the stored procedures code, making things a bit more difficult to maintain.
- **Migrating** to a different database management system (DB2, SQL Server, etc.) may potentially be more difficult.

Step 1 - Picking a Delimiter

The delimiter is the character or string of characters that you'll use to tell the mySQL client that you've finished typing in an SQL statement. For ages, the delimiter has always been a semicolon. That, however, causes problems, because, in a stored procedure, one can have many statements, and each must end with a semicolon. In this tutorial I will use “//”

Step 2 - How to Work with a Stored Procedure

Creating a Stored Procedure

```
DELIMITER //

CREATE PROCEDURE `p2` ()
LANGUAGE SQL
DETERMINISTIC
SQL SECURITY DEFINER
COMMENT 'A procedure'
BEGIN
    SELECT 'Hello World !';
END//
```

Step 2 - How to Work with a Stored Procedure

The first part of the statement creates the procedure. The next clauses defines the optional characteristics of the procedure. Then you have the name and finally the body or routine code.

Stored procedure names are case insensitive, and you cannot create procedures with the same name. **Inside a procedure body, you can't put database-manipulation statements(?).**

Step 2 - How to Work with a Stored Procedure

The four characteristics of a procedure are:

- **Language** : For portability purposes; the default value is SQL.
- **Deterministic** : If the procedure always returns the same results, given the same input. This is for replication and logging purposes. The default value is NOT DETERMINISTIC.
- **SQL Security** : At call time, check privileges of the user. INVOKER is the user who calls the procedure. DEFINER is the creator of the procedure. The default value is DEFINER.
- **Comment** : For documentation purposes; the default value is ""

Step 2 - How to Work with a Stored Procedure

Calling a Stored Procedure

To call a procedure, you only need to enter the word CALL, followed by the name of the procedure, and then the parentheses, including all the parameters between them (variables or values). Parentheses are compulsory.

Example:

```
CALL stored_procedure_name (param1, param2, ....)
```

```
CALL procedure1(10, 'string parameter', @parameter_var);
```

Step 2 - How to Work with a Stored Procedure

Modify a Stored Procedure

MySQL provides an ALTER PROCEDURE statement to modify a routine, but only allows for the ability to change certain characteristics. If you need to alter the body or the parameters, you must drop and recreate the procedure.

Step 2 - How to Work with a Stored Procedure

Delete a Stored Procedure

Example:

```
DROP PROCEDURE IF EXISTS p2;
```

This is a simple command. The IF EXISTS clause prevents an error in case the procedure does not exist.

Step 3 - Parameters

Let's examine how you can define parameters within a stored procedure.

- CREATE PROCEDURE proc1 () : Parameter list is empty
- CREATE PROCEDURE proc1 (IN varname DATA-TYPE) : One input parameter. The word IN is optional because parameters are IN (input) by default.
- CREATE PROCEDURE proc1 (OUT varname DATA-TYPE) : One output parameter.
- CREATE PROCEDURE proc1 (INOUT varname DATA-TYPE) : One parameter which is both input and output.

Of course, you can define multiple parameters defined with different types.

Step 3 - Parameters

IN example

```
DELIMITER //

CREATE PROCEDURE `proc_IN` (IN var1 INT)
BEGIN
    SELECT var1 + 2 AS result;
END//
```

Step 3 - Parameters

OUT example

```
DELIMITER //

CREATE PROCEDURE `proc_OUT` (OUT var1 VARCHAR(100))
BEGIN
    SET var1 = 'This is a test';
END //
```

Step 3 - Parameters

INOUT example

```
DELIMITER //

CREATE PROCEDURE `proc_INOUT` (INOUT var1 INT)
BEGIN
    SET var1 = var1 * 2;
END //
```

Step 4 - Variables

The following step will teach you how to define variables, and store values inside a procedure. You must declare them explicitly at the start of the BEGIN/END block, along with their data types. Once you've declared a variable, you can use it anywhere that you could use a session variable, or literal, or column name.

Declare a variable using the following syntax:

DECLARE varname DATA-TYPE DEFAULT defaultvalue;

Step 4 - Variables

Examples:

```
DECLARE a, b INT DEFAULT 5;

DECLARE str VARCHAR(50);

DECLARE today TIMESTAMP DEFAULT CURRENT_DATE;

DECLARE v1, v2, v3 TINYINT;
```

Step 4 - Variables

Working with variables

Once the variables have been declared, you can assign them values using the SET or SELECT command:

```
DELIMITER //
CREATE PROCEDURE `var_proc` (IN paramstr VARCHAR(20))
BEGIN
   DECLARE a, b INT DEFAULT 5;
   DECLARE str VARCHAR(50);
   DECLARE today TIMESTAMP DEFAULT CURRENT_DATE;
   DECLARE v1, v2, v3 TINYINT;

   INSERT INTO table1 VALUES (a);
   SET str = 'I am a string';
   SELECT CONCAT(str,paramstr), today FROM table2 WHERE b >=5;
END //
```

Step 5 - Flow Control Structures

MySQL supports the IF, CASE, ITERATE, LEAVE LOOP, WHILE and REPEAT constructs for flow control within stored programs. We're going to review how to use IF, CASE and WHILE specifically, since they happen to be the most commonly used statements in routines.

Step 5 - Flow Control Structures (IF)

```
DELIMITER //
CREATE PROCEDURE `proc_IF` (IN param1 INT)
BEGIN
    DECLARE variable1 INT;
    SET variable1 = param1 + 1;

    IF variable1 = 0 THEN
        SELECT variable1;
    END IF;

    IF param1 = 0 THEN
        SELECT 'Parameter value = 0';
    ELSE
        SELECT 'Parameter value <> 0';
    END IF;
END //
```

Step 5 - Flow Control Structures (CASE)

```
DELIMITER //
CREATE PROCEDURE `proc_CASE` (IN param1 INT)
BEGIN
    DECLARE variable1 INT;
    SET variable1 = param1 + 1;

    CASE variable1
        WHEN 0 THEN
            INSERT INTO table1 VALUES (param1);
        WHEN 1 THEN
            INSERT INTO table1 VALUES (variable1);
        ELSE
            INSERT INTO table1 VALUES (99);
    END CASE;

END //
```

Step 5 - Flow Control Structures (CASE)

```
DELIMITER //
CREATE PROCEDURE `proc_CASE` (IN param1 INT)
BEGIN
    DECLARE variable1 INT;
    SET variable1 = param1 + 1;

    CASE variable1
        WHEN 0 THEN
            INSERT INTO table1 VALUES (param1);
        WHEN 1 THEN
            INSERT INTO table1 VALUES (variable1);
        ELSE
            INSERT INTO table1 VALUES (99);
    END CASE;

END //
```

Step 5 - Flow Control Structures (CASE)

```
DELIMITER //
CREATE PROCEDURE `proc_CASE` (IN param1 INT)
BEGIN
    DECLARE variable1 INT;
    SET variable1 = param1 + 1;

    CASE
        WHEN variable1 = 0 THEN
            INSERT INTO table1 VALUES (param1);
        WHEN variable1 = 1 THEN
            INSERT INTO table1 VALUES (variable1);
        ELSE
            INSERT INTO table1 VALUES (99);
    END CASE;

END //
```

Step 5 - Flow Control Structures (WHILE)

```
DELIMITER //

CREATE PROCEDURE `proc_WHILE` (IN param1 INT)
BEGIN
    DECLARE variable1, variable2 INT;
    SET variable1 = 0;

    WHILE variable1 < param1 DO
        INSERT INTO table1 VALUES (param1);
        SELECT COUNT(*) INTO variable2 FROM table1;
        SET variable1 = variable1 + 1;
    END WHILE;
END //
```

Step 6 - Cursors

Cursor is used to iterate through a set of rows returned by a query and process each row.

```
DECLARE cursor-name CURSOR FOR SELECT ...;
/*Declare and populate the cursor with a SELECT statement */
DECLARE CONTINUE HANDLER FOR NOT FOUND
/*Specify what to do when no more records found*/
OPEN cursor-name;
/*Open cursor for use*/
FETCH cursor-name INTO variable [, variable];
/*Assign variables with the current column values*/
CLOSE cursor-name;
/*Close cursor after use*/
```

Step 6 - Cursors

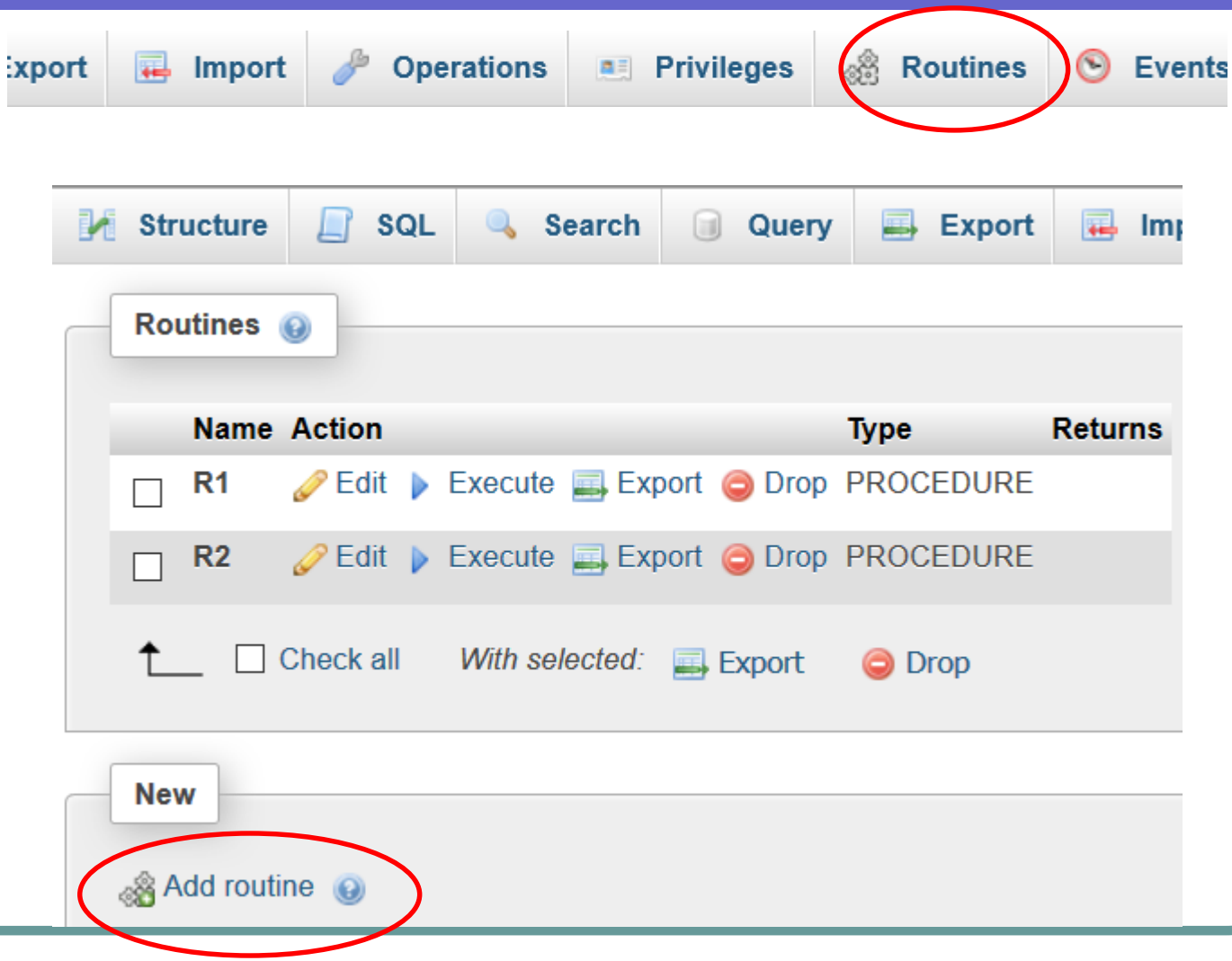
```
DELIMITER //

CREATE PROCEDURE `proc_CURSOR` (OUT param1 INT)
BEGIN
   DECLARE a, b, c INT;
   DECLARE cur1 CURSOR FOR SELECT col1 FROM table1;
   DECLARE CONTINUE HANDLER FOR NOT FOUND SET b = 1;
   OPEN cur1;

   SET b = 0;
   SET c = 0;

   WHILE b = 0 DO
      FETCH cur1 INTO a;
      IF b = 0 THEN
         SET c = c + a;
   END IF;
   END WHILE;

   CLOSE cur1;
   SET param1 = c;

END //
```

Step 6 - Cursors

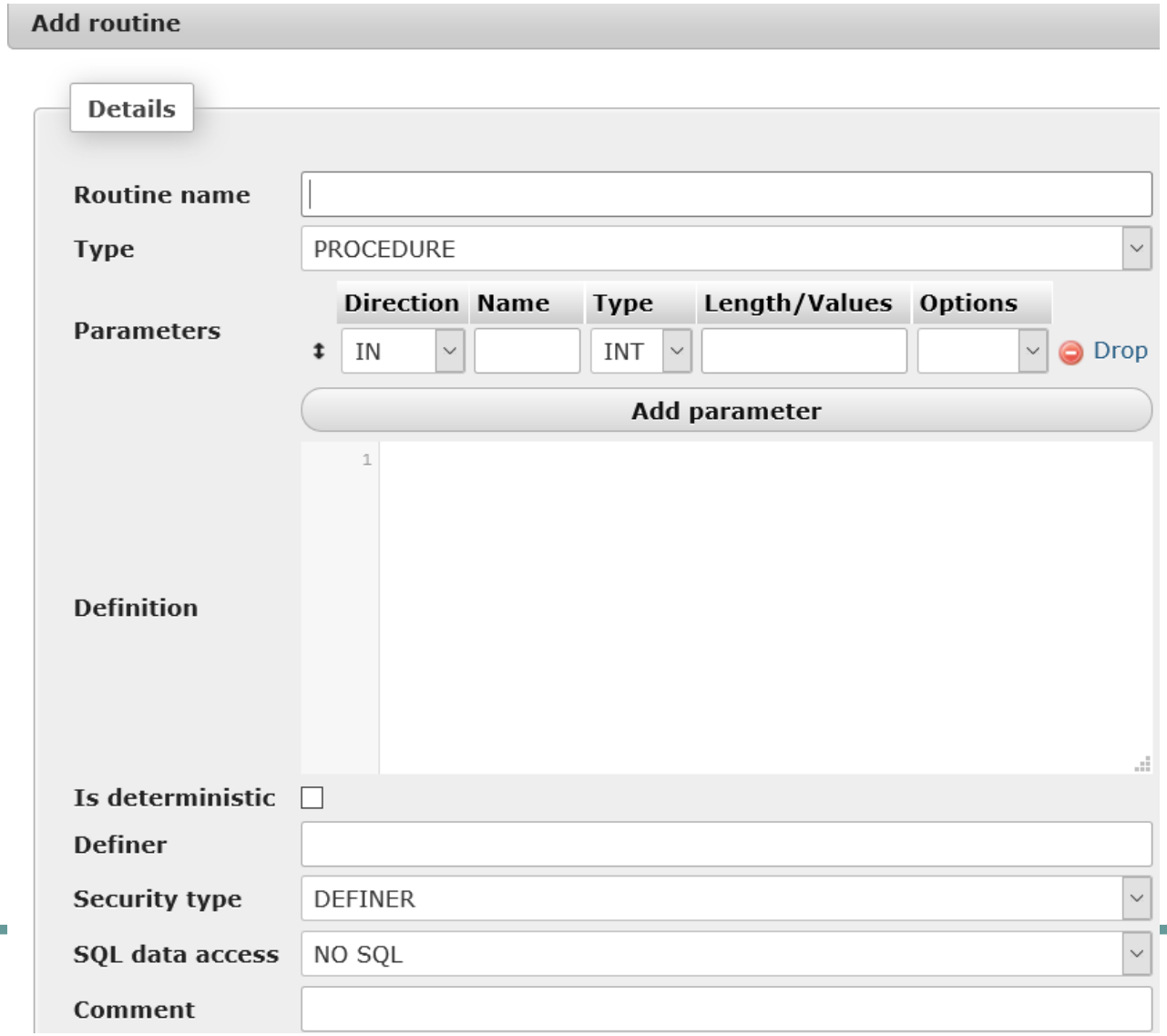
Cursor has three important properties that you need to be familiar with in order to avoid unexpected results:

- **Asensitive** : Once open, the cursor will not reflect changes in its source tables. In fact, MySQL does not guarantee the cursor will be updated, so you can't rely on it.
- **Read Only** : Cursors are not updatable.
- **Not Scrollable** : Cursors can be traversed only in one direction, forward, and you can't skip records from fetching.

PROCEDURES


Export
Import
Operations
Privileges
Routines
Events
Structure
SQL
Search
Query
Export
Imp
Routines
Name Action Type Returns
R1 Edit Execute Export Drop PROCEDURE
R2 Edit Execute Export Drop PROCEDURE
Check all With selected: Export Drop
New
Add routine

PROCEDURES


Add routine
Details
Routine name
Type
PROCEDURE
Parameters
Direction
Name
Type
Length/Values
Options
IN
INT
Drop
Add parameter
Definition
1
Is deterministic
Definer
Security type
DEFINER
SQL data access
NO SQL
Comment